SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: MLA03
COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10 Total Marks: 50

Annexure A
CODING CHALLENGE Duration: 2hrs

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. (5 marks)

Answer:

Reinforcement Learning (RL) is a framework in which an agent learns to make decisions by interacting continuously with an environment. At each time step, the agent observes the current state of the environment, selects an action based on its policy, and the environment responds by transitioning to a new state and returning a reward signal. This closed loop — state, action, reward, next state — repeats until the task ends, forming the agent–environment interaction cycle.

States represent the information available to the agent about the environment at a given moment, and they form the basis on which decisions are made. Actions are the choices available to the agent that influence how the environment evolves. Rewards are scalar feedback signals that indicate how good or bad an action was in a given state, guiding the agent toward desirable behaviour.

The agent's ultimate objective is not to maximize the immediate reward alone but the cumulative reward, often expressed as the discounted sum of future rewards. This long-term perspective forces the agent to weigh short-term gains against long-term benefits, encouraging strategies (policies) that may sacrifice small immediate rewards for larger rewards later. Cumulative reward therefore acts as the central optimization target that shapes all of the agent's decision-making.

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. (5 marks)

Answer:

Sequential decision-making problems under uncertainty are naturally modeled using a Markov Decision Process (MDP), formally defined by the tuple (S, A, P, R, γ). The MDP formulation assumes the Markov property: the next state and reward depend only on the current state and action, not on the full history, which makes the problem mathematically tractable.

- Action space (A): the complete set of actions available to the agent in each state; it can be discrete (e.g., move up/down/left/right) or continuous (e.g., steering angle), and defines what choices the agent can make at every decision point.
- Transition probability (P): denoted P(s'|s,a), it specifies the probability of moving to next state s' given that action a was taken in state s. It captures the stochastic dynamics of the environment. ● Reward function (R): denoted R(s,a,s'), it assigns a numerical reward for taking action a in state s and arriving in state s'; it encodes the goal of the task.
- Discount factor (γ): a value between 0 and 1 that determines how much future rewards are weighted relative to immediate rewards, ensuring the cumulative return is finite and controlling the agent's foresight.

By formalizing a problem into these five components, any sequential decision task — from robot navigation to game playing — can be estimated and solved using MDP-based RL algorithms such as value iteration, policy iteration, or Q-learning.

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. (5 marks)

Answer:

A policy, π, defines the agent's behaviour by mapping states to actions (or to a probability distribution over actions). It is essentially the strategy the agent follows while interacting with the environment, and the goal of RL is to find an optimal policy π* that maximizes expected cumulative reward.

Value functions estimate how good it is to be in a given state (or to take a given action in a state) under a particular policy, providing a way to evaluate and compare policies without having to simulate entire trajectories. The state-value function $V\pi(s)$ gives the expected cumulative return starting from state s and following policy π thereafter. The action-value function $Q\pi(s,a)$ gives the expected cumulative return starting from state s, taking action a, and following policy π afterward.

Both functions are computed recursively using the Bellman Equation, which expresses the value of a state (or state-action pair) in terms of the immediate reward plus the discounted value of the successor state. This recursive decomposition allows the agent to propagate long-term consequences backward through the state space, enabling algorithms like value iteration and Q-learning to iteratively refine value estimates. Because $Q\pi(s,a)$ directly compares the outcome of different actions in the same state, it is especially useful for deriving a greedy or improved policy — the action with the highest Q-value is preferred, driving policy improvement toward optimality.

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ε greedy and softmax approaches in balancing learning efficiency. (5 marks)

Answer:

The exploration–exploitation trade-off arises because an RL agent must balance two competing needs: exploiting the best-known action to maximize immediate reward, and exploring lesser-known actions to discover potentially better rewards. Exploiting exclusively risks converging to a suboptimal policy because the agent never learns about better alternatives, while exploring excessively wastes time on poor actions and slows convergence. A good balance is essential for the agent to learn an accurate and eventually optimal policy in a reasonable amount of time.

The ε-greedy strategy addresses this by choosing the currently best-known action with probability $(1 - \varepsilon)$ and a random action with probability ε. It is simple to implement and tune, but treats all non-greedy actions equally regardless of how promising they appear, and typically requires ε to be decayed over time to shift from exploration toward exploitation as learning progresses.

The softmax (Boltzmann) strategy instead assigns each action a selection probability proportional to its estimated value, scaled by a temperature parameter. Actions with higher estimated value are chosen more often, but weaker actions still retain a non-zero, value-proportional chance of being tried. This makes softmax more informed than ε-greedy since it discriminates between clearly bad and moderately good alternatives, generally leading to smoother, more efficient learning, though it introduces the added complexity of tuning the temperature parameter.

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. (5 marks)

Answer:

Reward shaping is the technique of supplementing an environment's sparse or delayed reward signal with additional, carefully designed intermediate rewards that guide the agent more directly toward the desired goal. In many real-world tasks, the natural reward is given only at the end of an episode (e.g., win/lose), which makes learning slow because the agent receives little feedback about which of its many actions actually contributed to success.

By providing denser feedback — for example, rewarding a robot for reducing its distance to a target, or a game agent for collecting useful items — reward shaping helps the agent associate specific actions with progress toward the goal much earlier in training. This significantly speeds up convergence because the agent does not have to rely purely on random exploration to stumble upon the sparse terminal reward.

However, poorly designed shaping rewards can inadvertently create suboptimal policies by allowing the agent to exploit the shaping signal instead of the true objective (a phenomenon sometimes called reward hacking). To prevent this, shaping rewards should be designed using principled approaches such as potential-based reward shaping, which mathematically guarantees that the optimal policy of the shaped reward function remains the same as that of the original reward function. When applied correctly, reward shaping accelerates learning while preserving policy optimality, striking a balance between faster training and correct long-term behaviour.

Code of Conduct:

I K.V.S.S. JESWANTH BABU (192472012) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary

action. ***Signature of the student:***

RUBRICS FOR CALCULATION

Criteria		**Weightage Level 4 (**		**Level 2 (De**	
Technical Knowledg 2		Demonstrates strong and accurate subject knowledge with in depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real	Applies concepts with moderate accuracy	Limited problem solving ability; requires guidance	Unable to solve or apply concepts

		scenarios			
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Question 1		Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately